

PROJECT AND TEAM INFORMATION

Algorithmic Trading and Market Simulation Platform

Student / Team Information

Team Name: Team ID:	TradeSim DSCPP-III-2026- T331
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Jatrana, Satvik – 2510019213 2510019213@geu.ac.in 
Team member 2 (Last Name, name: student ID: email, picture):	Bhatt, Siddharth – 2510012106 2510012106@geu.ac.in 
Team member 3 (Last Name, name: student ID: email, picture):	Uniyal, Akshat – student ID email ID picture
Team member 4 (Last Name, name: student ID: email, picture):	Kumar, Sourabh – student ID email ID picture

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Financial markets handle a large number of buy and sell orders, and these orders have to be organized and processed in the right order. A trading system needs to decide which orders should be considered first and when two orders can be matched. This makes trading systems a good real-world example of how data structures and algorithms can be applied to solve practical problems.

Many basic trading applications allow users to buy and sell assets or view prices, but the actual process happening behind the interface is usually hidden. The way an order book stores orders, maintains priority, and matches buyers with sellers is an important part of a trading system.

*Through this project, we plan to build a simplified market simulation called **TradeSim**. It will allow virtual traders to place buy and sell orders, maintain an order book, match compatible orders, record trades, and update virtual portfolios.*

*The project will help us apply **Data Structures in C** and **Object-Oriented Programming with C++** to one practical application. It will also give us a foundation for future extensions such as historical market data, strategy testing, backtesting, and quantitative analysis.*

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

In real financial markets, electronic trading systems receive buy and sell orders and process them according to predefined rules. These orders are maintained in an order book, where price and time are important for deciding which order gets priority. A matching system then looks for compatible buy and sell orders and generates trades when the required conditions are satisfied.

Most traders interact with these systems through stock exchanges or brokerage platforms, while the internal order-management and matching process remains hidden from them.

Educational trading applications generally provide features such as virtual portfolios, price tracking, transaction history, and simulated buying or selling. These applications are useful for understanding basic trading, but they may not clearly show the data structures and algorithms working behind the trading process.

***TradeSim focuses on this underlying process.** Instead of connecting to a real exchange, the project will create a controlled simulation using C/C++. This will allow us to demonstrate how orders are stored, prioritized, matched, recorded, and reflected in virtual portfolios.*

Project Goals and Milestones (2 pts)

General Goals:

- Develop a simple and reliable **C/C++ market simulation** where virtual traders can place and manage buy and sell orders.
- Maintain a clear **order book** that organizes pending buy and sell orders according to price and time priority.
- Implement a **matching engine** that identifies compatible orders and supports both complete and partial order execution.
- Maintain **virtual traders and portfolios** by recording trades, updating holdings, and tracking available virtual balance.
- Apply suitable **data structures and algorithms** for storing, searching, sorting, and processing orders efficiently.
- Use **C++ OOP concepts** to divide the system into clear components such as *Order*, *Trader*, *OrderBook*, *MatchingEngine*, *Trade*, and *Portfolio*.
- Provide **file-based storage** for maintaining important order, trade, and portfolio records.
- Test the system using normal, invalid, boundary, cancellation, complete-fill, and partial-fill cases.
- Keep the system open for future extensions such as **historical market data, trading strategies, backtesting, and quantitative analysis**.

Milestones:

- **M1 (Weeks 1–2): Requirement Analysis & C Data Structures:** Finalize the project requirements and trading workflow. Design the *Order* structure and implement the basic data structures required for storing and managing orders. Develop the initial console interface.
- **M2 (Week 3): Order Book & Order Management:** Develop separate buy and sell order books. Implement order insertion, cancellation, validation, and basic price-time priority handling.
- **M3 (Week 4): Matching Engine:** Develop the main **price-time priority matching algorithm**. Implement compatible order detection and handle both complete and partial order execution.
- **M4 (Week 5): C++ OOP Layer & Class Integration:** Develop the major components as C++ classes such as *Order*, *Trader*, *OrderBook*, *MatchingEngine*, *Trade*, and *Portfolio*. Apply encapsulation, abstraction, constructors, and suitable access control.
- **M5 (Week 6): Trade & Portfolio Management:** Connect the matching engine with the portfolio module. Record executed trades, update virtual cash and holdings, and calculate basic profit and loss.
- **M6 (Week 7): File Persistence & Testing:** Implement file input/output for saving and loading important records. Test valid and invalid orders, order cancellation, price-time priority, complete fills, partial fills, and boundary cases.
- **M7 (Final Phase): Integration, Demonstration & Documentation:** Combine all modules into a stable working application. Perform end-to-end testing, prepare test results and screenshots, document the implemented data structures and algorithms, and finalize the project report and presentation.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

The project will be developed as a modular C/C++ application. **C will be used for implementing and demonstrating fundamental data structures and algorithms**, while C++ will be used to organize the complete system using object-oriented programming.

The system will contain components such as **Order, Trader, OrderBook, MatchingEngine, Trade, and Portfolio**. A trader will first submit a buy or sell order. The order-management module will check the input and place the valid order on the appropriate side of the order book.

The **matching engine** will examine the best available buy and sell orders. When their prices satisfy the matching condition, a trade will be created. The system will also support **partial execution**, where only part of an order is matched and the remaining quantity stays in the order book.

Executed trades will be recorded and used to update the virtual portfolios of the participating traders. Initially, file-based storage will be used for maintaining important simulation records.

The project will use C, C++, suitable Standard Template Library components, a C/C++ compiler and IDE, and Git for version control. Development will be carried out step by step through **design, implementation, testing, debugging, and integration**. Testing will cover valid and invalid inputs, order cancellation, price-time priority, complete fills, and partial fills.

Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The main outcome of the project will be a working **C/C++ market simulation** in which virtual traders can place buy and sell orders and observe how those orders are processed.

The completed system will maintain an **order book**, match compatible orders, record executed trades, and update virtual portfolios. It will demonstrate how data structures and algorithms can be applied to organize and process trading orders, while C++ OOP will be used to build the different components of the system.

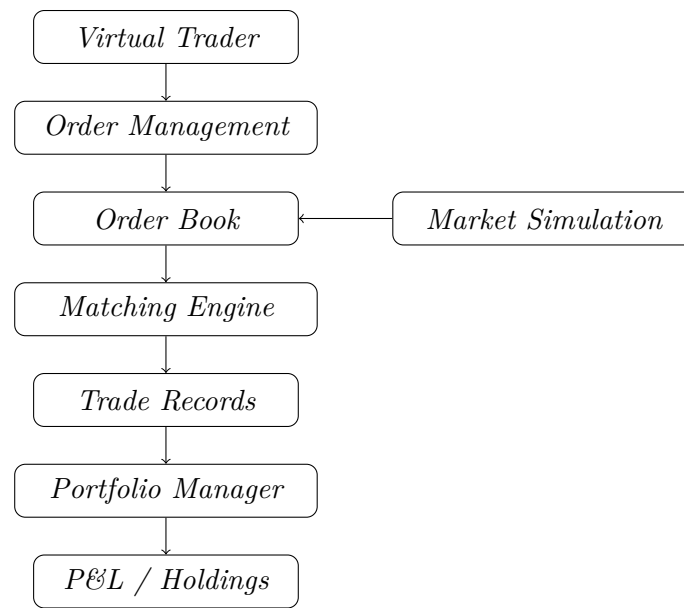
The main deliverables will include:

- **Working TradeSim application** with virtual trading functionality.
- **Order book and matching engine** with complete and partial order execution.
- **Implemented data structures and algorithms** with suitable documentation.
- **Trade and portfolio records** with basic P&L tracking.
- **Test cases and results** covering different trading situations.
- **Source code, project report, and presentation.**

The system will also provide a foundation for future work involving **historical market data, trading strategies, backtesting, and quantitative analysis**.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



High-Level Architecture of the TradeSim Market Simulation

The system starts with a **virtual trader** submitting a buy or sell order. The order-management module validates the order and sends it to the appropriate side of the **order book**. The order book keeps pending orders organized according to their priority.

The **matching engine** checks the best available buy and sell orders. When the matching conditions are satisfied, the required quantity is executed and a trade is created. The trade is then recorded and the portfolios of the participating traders are updated.

The **market simulation module** can generate controlled orders and different trading situations, allowing the system to be tested under different conditions.

Subject Integration

Data Structures in C:

- **Structures:** Used to represent an order by storing information such as order ID, trader ID, price, quantity, order type, and timestamp.
- **Arrays:** Used where fixed-size collections of orders, traders, or records need to be stored and accessed.
- **Linked Lists:** Used for dynamically managing pending orders, where orders may be added or removed during the simulation.
- **Queues:** Used to maintain the time order of orders placed at the same price level, so that earlier orders receive priority.
- **Searching and Sorting:** Used to find relevant orders and organize buy and sell orders according to their priority in the order book.

OOPs with C++:

- **Classes and Objects:** Used to represent the main components of the system, including **Order**, **Trader**, **OrderBook**, **MatchingEngine**, **Trade**, and **Portfolio**.
- **Encapsulation:** Used to keep the data and operations of each component together and control how the data can be modified.
- **Abstraction:** Used to hide internal processing and provide simple operations such as submitting an order, matching orders, and updating a portfolio.
- **Constructors and Destructors:** Used to initialize objects when they are created and manage their resources during their lifetime.
- **Exception Handling:** Used to handle invalid inputs and incorrect orders without causing the complete application to terminate unexpectedly.

Integration:

The two subjects come together in the main workflow of TradeSim. **Data structures manage how orders are stored, organized, and processed**, while C++ classes organize these operations into separate and reusable components. Together, they form the complete flow from order submission and order-book management to matching, trade recording, and portfolio updates.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

- The project will use a **simulated market** with virtual money and will not connect to a real stock exchange.
- Orders will be generated by virtual traders or by controlled market-simulation data.
- The project will focus on the **core order-management and matching process** rather than reproducing the full complexity of a production trading system.
- File-based storage will initially be used for saving relevant orders, trades, and portfolio records.
- Historical data, trading strategies, and backtesting will be considered **future extensions** rather than essential parts of the initial PBL implementation.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. *cppreference.com, C++ Reference.*
<https://en.cppreference.com/>
2. *cppreference.com, C Language Reference.*
<https://en.cppreference.com/w/c>
3. *cppreference.com, C++ Standard Library Containers.*
<https://en.cppreference.com/w/cpp/container>
4. *Overleaf Documentation.*
<https://www.overleaf.com/learn>
5. *Course materials and lecture notes for Data Structures in C and OOPs with C++.*